

Задание №1.

Запросы на вливание

Цели задания:

- Изучить механизм проведения запросов на вливание (pull request) с использованием Visual Studio и GitHub.

В предыдущих заданиях мы научились использовать версионный контроль для хранения своего кода в удаленном репозитории. Механизм работы с версионным контролем предполагал создание удаленного репозитория на GitHub и клона на локальной машине. Все изменения в коде сохранялись в одной ветке develop. Такой подход ведения репозитория подходит для выполнения учебных заданий одного разработчика, но не подходит для командной разработки проекта, где с некоторой периодичностью необходимо выпускать релизы приложения.

В команде каждый разработчик занимается решением своих задач. Чаще всего, при добавлении новой функциональности требуется сделать не один коммит, а несколько. Кроме того, из-за невнимательности, коммит может содержать не компилируемый код или код, где часть функциональности намеренно отключена.

Если все разработчики будут отправлять код в одну ветку develop, то код одного разработчика может помешать работать всем остальным. Отсюда возникает правило – в основную ветку (develop) должен попадать код только законченной, работоспособной и проверенной функциональности.

Чтобы соблюдать это правило, и чтобы разработчики не мешали друг другу, работа с репозиторием должна быть организована через ветки. Каждый разработчик для каждой задачи создаёт новую ветку из ветки develop. В отдельной ветке разработчик может делать любое количество произвольных коммитов, ни как не влияя на работу других разработчиков. А другие разработчики не будут влиять на вашу работу. Когда разработчик завершит решение своей задачи, он должен проверить работу кода, правильность оформления и других требований, после чего сделать слияние своей ветки в ветку develop. В таком случае, в

develop попадёт уже готовая реализация, которая не мешает другим разработчикам.

Слияние веток

Для перемещения изменений из одной ветки в другую используются команды `merge` или `rebase`. Обе команды выполняют перемещение изменений, но с небольшим отличием. В случае команды `rebase` все коммиты вашей ветки переносятся в `develop` (или другую ветку), при этом с точки зрения истории репозитория всё будет выглядеть так, будто все коммиты изначально выполнялись в ветке `develop` – а вашей ветки никогда не существовало. В случае команды `merge` версионный контроль самостоятельно сделает ещё один коммит в ветку `develop`, куда зальёт изменения из вашей ветки. При этом ваша ветка останется в истории без изменений, и к ней можно будет вернуться.

Конечный результат команд не отличается – изменения будут перенесены из одной ветки в другую. Но в разных командах разработчиков предпочитают разные способы слияния веток. В одном случае (`rebase`) история репозитория будет линейной и аккуратной, удобной для восприятия, хотя часть информации о процессе разработки потеряется (кто и когда создавал, сливал ветки). В другом случае (`merge`) история репозитория хранится как есть, со всеми ошибками и промежуточными коммитами. Это выглядит более запутанным, но остается вся информация о работе команды в исходном коде.

Кроме того, у команд `merge` и `rebase` есть дополнительные параметры, которые влияют на способ слияния веток. Например, для команды `rebase` есть параметр `-squash`, который при слиянии веток все коммиты вашей ветки сожмёт в один коммит и переместит его в ветку `develop` (или другую целевую ветку). За счёт этого история репозитория будет выглядеть лаконично, где один коммит будет хранить полное решение какой-либо задачи. Минусом такого подхода является то, что объединенные коммиты будут хранить много изменений в исходном коде, что может затруднить поиск ошибок в репозитории, если таковые возникнут.

Однако, при работе в отдельных ветках недостаточно просто выполнить команду `merge` для своей ветки в ветку `develop`. Есть несколько важных правил при командной разработке:

- 1) В репозиторий должны попадать только компилируемые коммиты.
- 2) В ветку `develop` должна попадать код только полностью решенных задач, никаких промежуточных коммитов.

- 3) Разработка новых задач должна выполняться в отдельных ветках с последующим слиянием в develop.
- 4) Разработчики, кроме тимлида, не должны иметь прав на заливание своих коммитов напрямую в develop. Ветка develop должна быть защищена от прямой фиксации изменений.
- 5) Код в ветку develop должен попадать строго через проверку кода (code review). Есть проверяющий (reviewer) одобрит изменения в ветке с решенной задачей, ветку можно залить в develop. Иначе разработчик должен исправлять код, пока ревьюер его не одобрит.
- 6) Прежде чем отправить свой код на слияние в develop, сначала обновите свою локальную ветку develop и загрузите изменения из develop в свою ветку.

Правило 4 существует как раз для того, чтобы разработчики по ошибке не залили свой нерабочий код в ветку develop. Такое иногда случается, и это может остановить работу всей команды до тех пор, пока разработчик не исправит код.

Правило 5 обязывает проверять любой код, который отправляется в develop. Это даёт сразу несколько преимуществ:

- 1) Код младших разработчиков (junior) и новых участников команды, как правило, проверяется более опытными разработчиками. Замечания по живому коду позволяют быстрее обучить новых разработчиков.
- 2) Проверка кода вторым участником в разы уменьшает вероятность попадания нерабочего кода в основную ветку.
- 3) В результате проверки кода в команде как минимум два разработчика знают, как работает любой фрагмент кода. Тем самым повышается так называемый «bus factor». Ревьюеров на одну задачу может быть и больше одного, это решается командой.

Правило 5 реализуется с помощью механизма запросов на вливание (pull request). Как мы знаем, команда pull позволяет получить изменения из другого репозитория. Pull request аналогичен команде pull, только получение изменений должно быть одобрено с принимающей стороны. То есть, если код отправляется из локального репозитория в origin (удаленный исходный репозиторий), то кто-то из разработчиков с соответствующими правами должен одобрить вливание ваших изменений в origin. Об этом мы поговорим чуть подробнее дальше.

Правило 6 защищает ветку develop от конфликтов – ситуации, когда один и тот же фрагмент файла был изменен в разных ветках, и при слиянии версионный

контроль не смог автоматически объединить эти версии. Конфликты разрешаются разработчиками вручную, а работа с такой веткой невозможна до разрешения конфликта.

Как такая ситуация возникает и как правило 6 помогает избежать такой ситуации? Дело в том, что при командной разработке вы будете не одним разработчиком, который отправляет код в develop. Пока вы создали собственную ветку и решали свою задачу, кто-то из других разработчиков (вероятно, даже не один), уже закончил свою задачу и отправил их в develop оригинального репозитория.

Таким образом, возникает ситуация, что ваша ветка и ветка develop в вашем репозитории не содержат последних изменений из удаленного репозитория. Если по стечению обстоятельств вы в своей ветке поменяете тот же фрагмент кода, что и другой разработчик в своей ветке (которая уже залита в develop удаленного репозитория), то при отправке вашего кода в develop удаленного репозитория там возникнет конфликт, который остановит работу всех разработчиков при их первой же синхронизации с удаленным репозиторием.

Поэтому, после завершения своей задачи в своей ветке, вы должны:

- 1) переключиться в ветку develop
- 2) синхронизировать ветку develop с удаленным репозиторием, т.е. выкачать последние изменения из удаленного репозитория в свой локальный репозиторий.
- 3) переключиться обратно в ветку своей задачи.
- 4) выполнить слияние ветки develop в ветку вашей задачи – таким образом, изменения других разработчиков из удаленного репозитория попадут в вашу рабочую ветку. Слияние выполняется командой merge, но не rebase!
- 5) Если в вашей ветке после слияния возник конфликт, то разрешить его.
- 6) После разрешения конфликта отправить запрос на вливание (pull request). Запрос на вливание может выполняться как с merge, так и rebase.

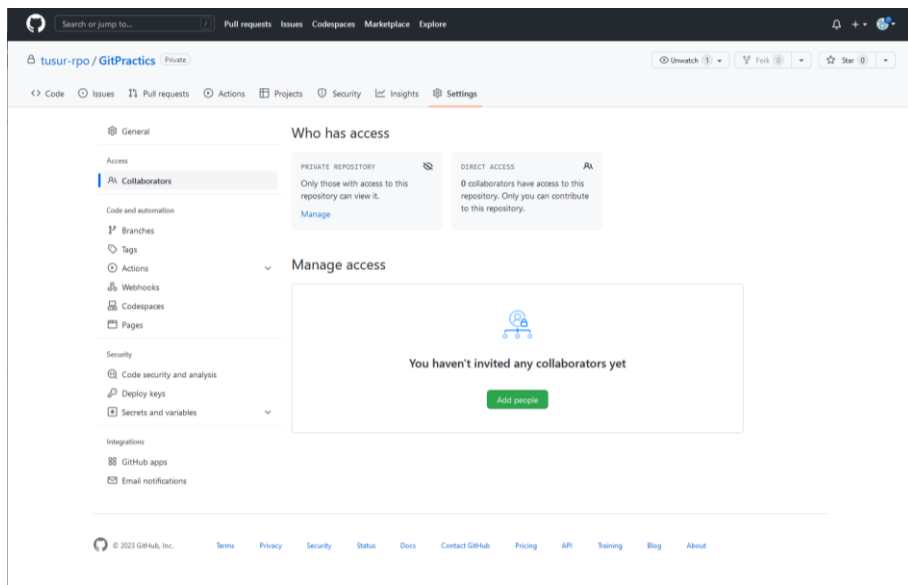
Возможно, вы заметили, что данная последовательность не защищает вас от конфликтов – если два разработчика поменяли один и тот же фрагмент кода в разных ветках, то конфликт всё равно произойдет. Но при описанном выше алгоритме конфликт возникнет в вашей ветке, а не в ветке develop удаленного репозитория. Таким образом, вы сможете поправить конфликт вашей и удаленной версии, не задерживая работу остальной команды. Последовательность

выше должна быть заучена вами и выполняться каждый раз при отправке своего кода.

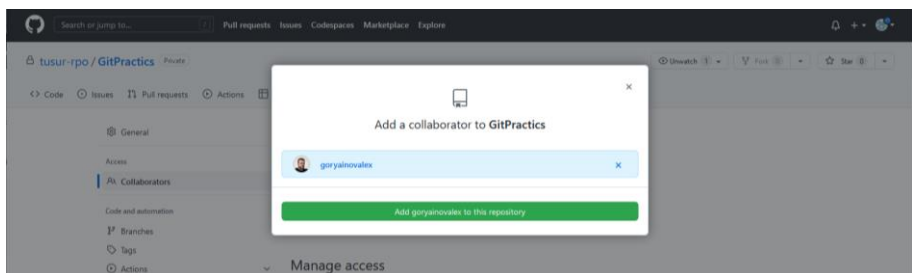
Также в пункте 4 указывается, что слияние из develop в вашу ветку должно выполняться только с помощью merge. Попробуйте ответить самостоятельно, почему в данном случае rebase использовать недопустимо.

Создание Pull Request

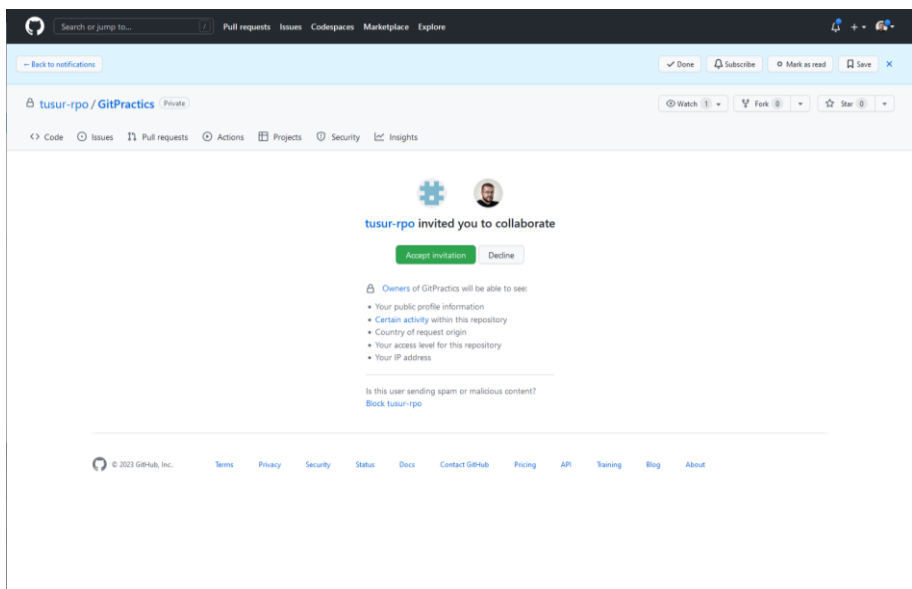
Создание Pull Request мы изучим с помощью инструментов GitHub. Как было сказано ранее, запросы на вливание предполагают, что запрос одобряет другой разработчик, который работает в этом же репозитории. Поэтому в первую очередь надо добавить еще одного участника к нашему репозиторию. Для этого необходимо зайти в настройки репозитория на GitHub и в панели Collaborators указать имя других пользователей GitHub:



Для выполнения задания можете завести второй аккаунт на GitHub или договориться с одноклассником и взаимно добавлении друг друга к репозиториям, чтобы изучить механику Pull Request. Укажите имя пользователя и нажмите Add:



После этого, пользователю будет отправлено приглашение в уведомления на GitHub и на почту, указанную при регистрации. Другому пользователю необходимо открыть и принять приглашение в уведомлениях:



Для изучения запросов на вливание предлагаем вам создать отдельный репозиторий, назвать его GitPractices и практиковаться в нём. Изучение веток и запросов на вливание на боевом репозитории, где у вас размещен рабочий код, может привести к нежелательным последствиям.

Теперь создадим ветку с изменениями, которые будут отправлены через запрос на вливание. Откройте (склонируйте) репозиторий на локальной машине, и перейдите в ветку develop. Вытяните (pull) все изменения из удаленного репозитория, а затем создайте новую ветку.

При работе в команде, все задачи ведутся в системе управления проектами. В таких системах каждая задача имеет своё название и свой уникальный номер. Чтобы не путаться в ветках и быстро их находить, рекомендуется следующее именование ветки:

```
features/n_taskname
```

где n – это номер задачи, а taskname – короткое, но понятное название задачи. Например:

```
features/2_learn_pull_requests  
features/14_add_discounts_to_customer  
features/67_fix_validation_on_customer_page
```

При таком именовании все ветки разработчиков будут лежать в подпапке features, что упростит навигацию по ним. При этом, так как номера задачам, как правило, присваиваются в хронологическом порядке, номер задачи обеспечит хронологическую сортировку веток. Короткое же название задачи позволяет быстрее понять, какая задача в этой ветке, вместо того чтобы открывать систему управления проектами и искать задачу по номеру. Если вы не будете использовать при именовании веток подпапку features, номер задачи или её название, то очень скоро вы запутаетесь в ветках. Чем больше команда разработчиков, тем больше веток будет создаваться.

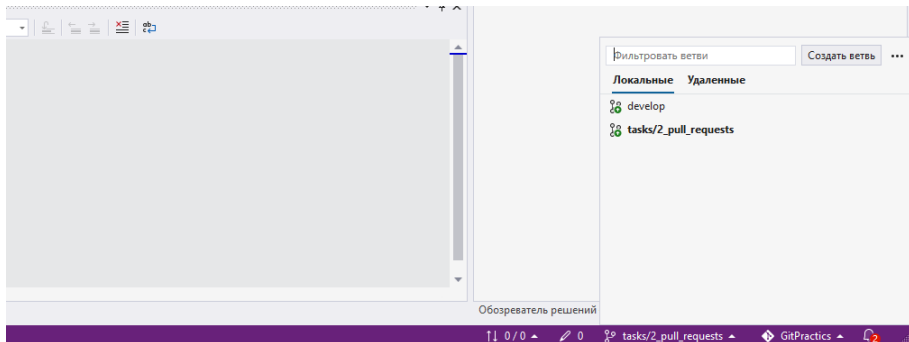
Кроме веток features вы также можете использовать именование tasks – главное, договоритесь в команде о едином именовании. Также обращайте внимание на верхние и нижние регистры в именовании веток, потому что features и Features – это разное именование и две разные подпапки. Ветки с исправлением ошибок, найденных тестировщиками или пользователями, рекомендуется складывать в подпапку bugs или fixes вместо папки features. Ветки с экспериментальной функциональностью или исследовательской кодовой базой (например, ветка для изучения сторонней библиотеки) рекомендуется также складывать в отдельную подпапку, например, research, learn или temp.

Создайте ветку с названием features/2_learn_pull_requests и внесите в неё любые изменения – добавьте пару текстовых файлов, измените существующий файл. Зафиксируйте изменения в ветке и синхронизируйтесь с удаленным репозиторием. Для новой созданной ветки команда pull будет недоступна, только push (отправить). Это объясняется просто – вы создали ветку в локальном репозитории и её пока не существует в удаленном. А если её нет в удаленном ре-

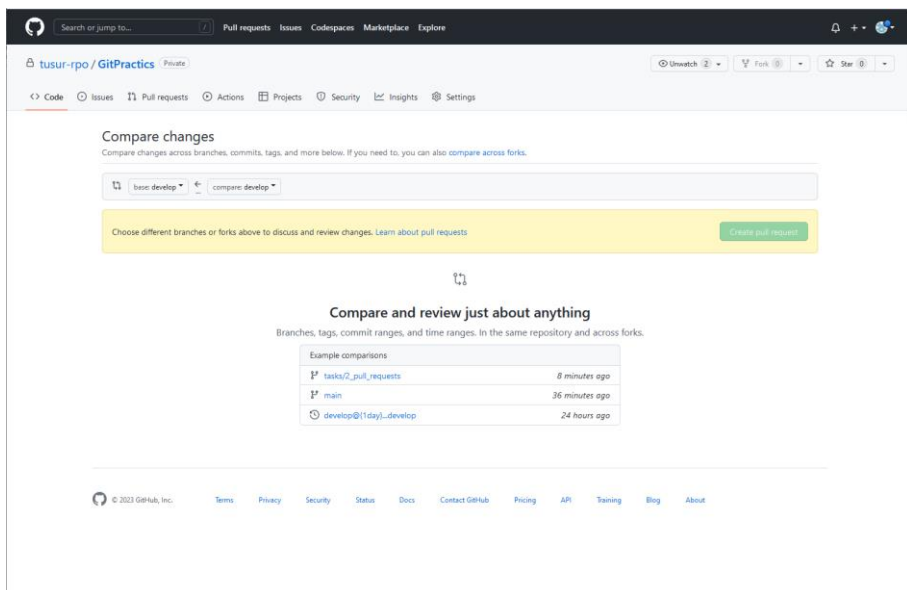
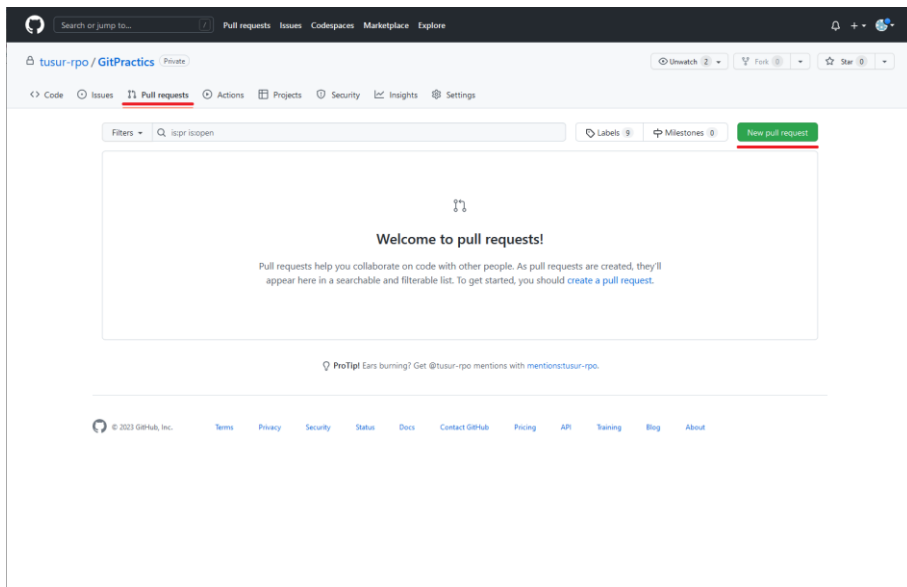
позитории, то и вытянуть (pull) изменения неоткуда. После того, как вы отправите ветку в удаленный репозиторий, команда pull для этой ветки станет доступной.

Теперь всё готово к отправке запроса на вливание. Выполните следующую последовательность действий:

- 1) Перейдите в ветку develop и синхронизируйтесь с удаленным репозиторием.
- 2) Вернитесь в свою рабочую ветку и слейте в неё изменения из ветки develop. Для этого откройте список веток через строку состояния в Visual Studio, как это показано на рисунке ниже. Текущая ветка выделена жирным. Правый клик по названию ветки develop и выбор пункта «Merge with Current Branch» («Слияние с Current Branch»).
- 3) Если обнаружались конфликты, то разрешите их. Если вы не делали коммитов в других ветках, кроме рабочей, то конфликтов не будет.

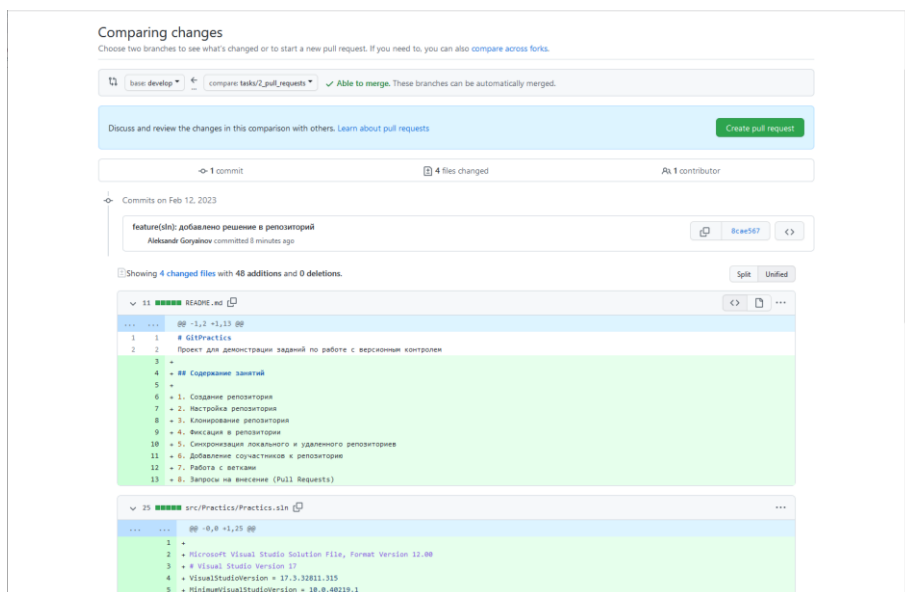


Будьте внимательны – на данном этапе надо слить изменения из develop в рабочую ветку, а не наоборот. Если вы всё сделали правильно, то переходите к окну GitHub. Из вкладки Code репозитория перейдите на вкладку Pull Request (см. рисунок ниже), а затем нажмите кнопку «New pull request». После этого в появившемся ниже кнопки списке ветку, которую вы хотите отправить на включение в develop.



Появится окно, в котором вы сможете предварительно сравнить рабочую ветку с веткой `develop`. Обязательно просмотрите все изменения, которые вы хотите отправить. Очень часто на этом этапе обнаруживается, что под версионный

контроль не добавились нужные файлы или, наоборот, добавились лишние. Здесь же можно увидеть ошибки в оформлении кода, отсутствие комментариев и другие мелочи. Обязательно перед созданием pull request просмотрите все изменения и исправьте очевидные ошибки сразу. Если этого не сделаете вы, тогда ревьюеру понадобится больше времени на проверку вашего кода, и принятие pull request затянется. Не отправляйте на ревью код с очевидными замечаниями. Если всё хорошо, нажмите кнопку «Create pull request» в верхней части экрана.



В следующем окне вы должны дать название и описание для Pull Request, а также назначить ревьюера. Название запроса рекомендуется делать по аналогии с именованием коммитов, но с некоторыми отличиями:

`N type(location):taskdescription`

где N – номер задачи, отправляемой на проверку. Если несколько задач, то перечислите их через запятую.

type – тип решенной задачи – feature, fix, codestyle, ui, ci, refactor.

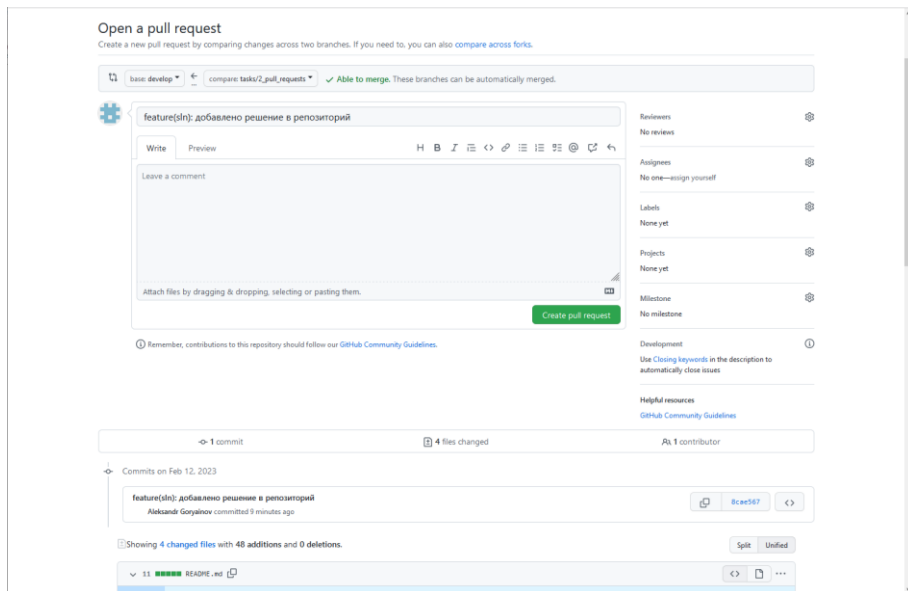
location – название класса, проекта или файла, которого коснулись ключевые изменения в запросе.

taskdescription – краткое описание задачи на русском языке.

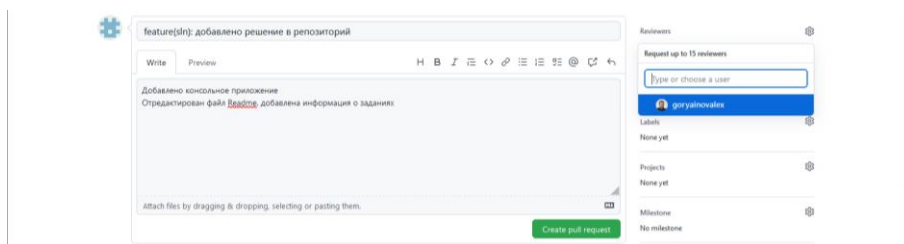
Например:

```
10 feature(customer): добавлена валидация данных пользователя
27 fix(order): добавлены зарезервированные ID для уникальных
заказов
55 codestyle(Model): исправлены грамматические ошибки в проекте
бизнес-логики, добавлены недостающие комментарии
```

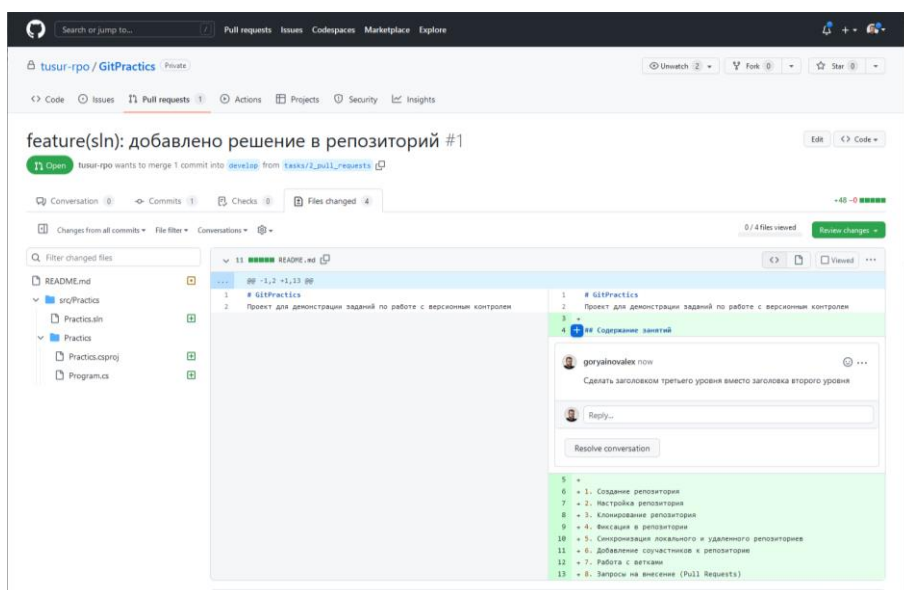
Если при именовании коммитов вы придерживаетесь ранее изученного формата, то именование PR для вас не составит сложности. Описание содержит более подробную информацию об изменениях в PR. Здесь же можно указать, с каких классов ревьюеру стоит начать проверять PR, объяснить логику взаимодействия новых классов, прикрепить скриншот элементов управления, если в задача связана с исправлением верстки пользовательского интерфейса. Также может возникнуть ситуация, когда один разработчик посылает новый PR до того, как предыдущий был принят или проверен. В таком случае в описании можно указать, что этот PR должен быть принят строго после предыдущего, так как включает в себя изменения из ветки другой задачи. Другими словами, напишите в описании любую информацию, которая упростит понять ваш код и быстрее его проверить.



Чтобы добавить ревьюера к PR, надо нажать значок шестеренки в правом меню напротив пункта Reviewers. В выпадающем списке отобразятся все пользователи, которых вы подключили к репозиторию. Политики репозитория на GitHub по умолчанию предполагают, что человек, создавший PR, не может его самостоятельно принять – всегда нужен другой разработчик. Затем нажмите кнопку «Create pull request».



После создания PR, ревьюеру придёт уведомление о том, что он добавлен к PR. Теперь ревьюер должен перейти в ваш репозиторий, открыть панель Pull Requests, выбрать нужный PR и проверить его. В частности, через панель File changed ревьюер сможет сравнить старые и новые версии файлов, где добавленный код будет подсвечен зеленым, а удаленный – красным.



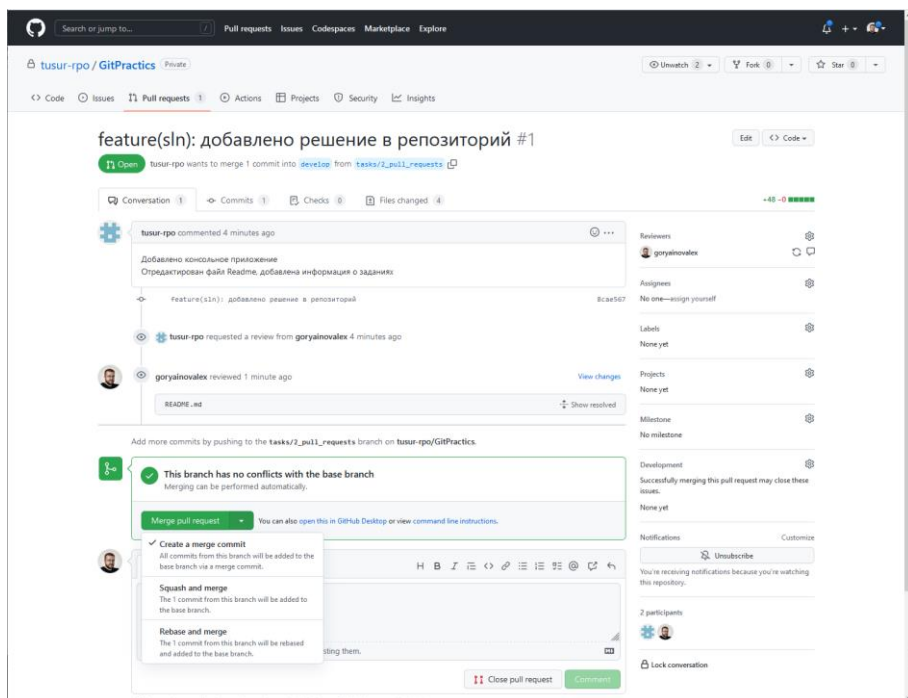
Подсветка строчек сильно упрощает проверку кода и позволяет сконцентрироваться только на новом коде, а не искать эти изменения самому.

Если ревьюер обнаружит какие-либо ошибки, то ему необходимо нажать синюю кнопку с плюсом напротив измененной строки кода и оставить замечание на исправление. После проверки ревьюер пишет разработчику, что оставил замечания, и разработчик также на панели Pull Requests может с ними ознакомиться.

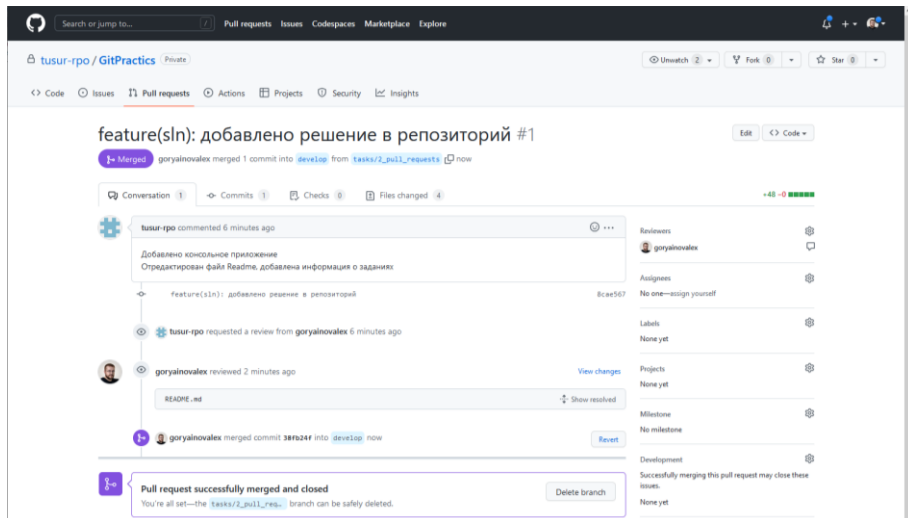
Теперь разработчику нужно исправить замечания. Для этого ему нужно открыть Visual Studio и залить исправления в рабочую ветку. Пересоздавать PR после исправления не нужно. Достаточно синхронизировать свою рабочую ветку с удаленным репозиторием – все новые исправления тут же отобразятся на GitHub.

После исправлений разработчик пишет ответный комментарий в каждом замечании, а затем сообщает об этом ревьюеру. Если замечание исправлено, ревьюер закрывает замечание нажатием кнопки «Resolve conversation». Если замечание исправлено неправильно, ревьюер пишет дополнение в комментариях и сообщает об этом разработчику.

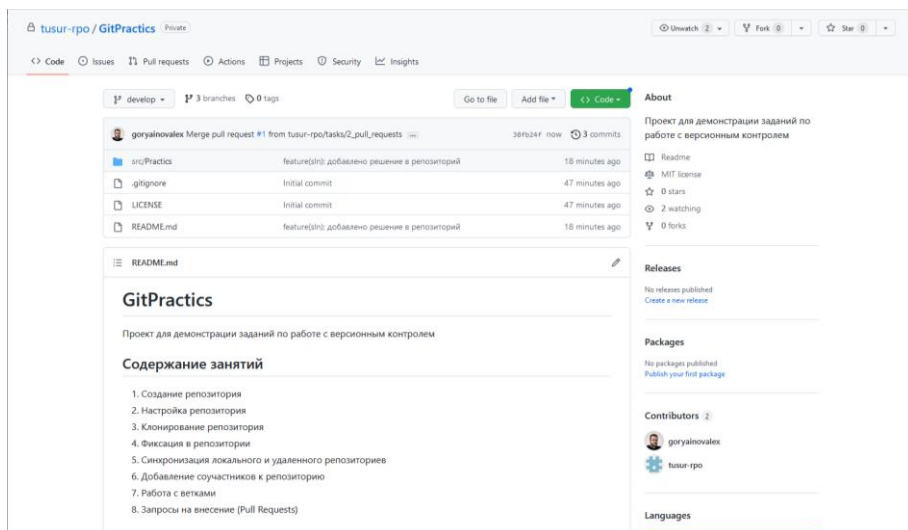
После того, как разработчик исправил все замечания, ревьюер закрывает PR. Для этого в окне Conversation надо нажать Merge pull request:



Здесь же можно выбрать способ для слияния веток – merge, rebase или squash rebase. По умолчанию используется merge, но это можно изменить в настройках репозитория, также, как и назначить ревьюера по умолчанию и другие политики. Если PR завершен успешно, то он будет отмечен как Merged:



Теперь можно перейти во вкладку Code и убедиться, что новые изменения действительно попали в ветку develop.



Заккрытие PR может быть отклонено системой, например, в случаях, если после слияния возникнет конфликт. В этом случае вам надо повторить обновление локальной ветки `develop` из удаленной и слияние её со своей рабочей веткой. Далее в рабочей ветке устранить конфликт, синхронизироваться с удаленным репозиторием и повторить попытку закрытия PR.

Задание

1. Создайте учебный репозиторий `GitPractics` на GitHub.
2. Создайте в нём ветку `develop`, сделайте её веткой по умолчанию.
3. Добавьте к репозиторию еще одного участника: свой второй аккаунт или одногруппника.
4. Склонируйте репозиторий на рабочий компьютер.
5. Создайте из ветки `develop` новую ветку.
6. Внесите изменения в репозиторий, добавьте несколько текстовых файлов или исправьте существующие (`readme`, `gitignore`).
7. Обновите локальную ветку `develop` из удаленного репозитория и залейте изменения из неё в новую рабочую ветку. Убедитесь, что нет конфликтов.
8. Через GitHub создайте Pull Request.
9. Через аккаунт ревьюера попробуйте добавить замечания в Pull Request, открывать и закрывать обсуждения замечаний.
10. Попробуйте добавить изменения в рабочую ветку в локальном репозитории и синхронизировать это с удаленным репозиторием. Убедитесь, что изменения появились на GitHub в окне просмотре Pull Request.
11. Закройте все созданные замечания и закройте Pull Request.
12. Обновите локальную ветку `develop` из удаленного репозитория.
13. Создайте еще одну ветку под новую задачу, внесите исправления и повторите процедуру Pull Request. Закрепите последовательность действий.
14. Убедившись, что процедура Pull Request для вас понятна, попробуйте повторить Pull Request в своём рабочем репозитории с предыдущими заданиями Programming.
15. К защите задания у вас должно быть не менее двух PR в репозитории `GitPractics` и хотя бы один PR в рабочем репозитории. Обращайте внимание на правильность именования репозитория, веток, PR и коммитов.

Дополнительное задание

1. Сделайте новую рабочую ветку и добавьте туда 3-4 коммита с изменениями. Сделайте PR и закройте его с командой `merge`. Затем повторите это для

команды rebase и squash. Откройте в Visual Studio историю ветки develop и посмотрите, чем отличается история ветки для каждого из трёх способов слияния веток. Для обычного merge в истории должно быть нарисовано ответвление из ветки develop, а затем снова слияние с ней. Для rebase все сделанные в новой ветке коммиты должны отображаться так, как будто они были сделаны непосредственно в develop, без ответвлений в истории. Для squash все коммиты из ветки должны превратиться в один коммит в develop. Покажите историю коммитов при защите задания.